

Dual-Mode OS Fingerprinting with Machine Learning: A Hybrid Approach for Network Enumeration

Mentor: Sruti Karumanchi (Project Lead, Cyber-Physical Systems Research)
Group Members: Aditya Yalamanchi, Abhijit Boro

Abstract—This research introduces an advanced OS fingerprinting tool employing a hybrid approach, combining passive fingerprinting via ARP spoofing and active fingerprinting through manual TCP connections. Powered by a Random Forest classifier, the machine learning model extracts features from network packets for accurate OS identification. The tool's modular design offers flexibility in selecting passive or active methods. Validated through real-world scenarios, the tool demonstrates precision in identifying OS types and versions, providing actionable insights for network security enhancement. The integration of machine learning ensures adaptability in dynamic network environments, making it a valuable asset for security professionals navigating intricate network landscapes.

I. INTRODUCTION

The relentless evolution of network technologies and the growing sophistication of cyber threats necessitate a robust and versatile approach to operating system (OS) fingerprinting. Accurate identification of the underlying OS on devices within a network is foundational to effective cybersecurity practices. This paper presents a pioneering OS fingerprinting framework that amalgamates both passive and active techniques, seamlessly integrated with a powerful machine learning model.

In our pursuit of a unified solution, we introduce a dual-pronged strategy. The passive component harnesses the subtlety of ARP spoofing, allowing for the non-intrusive capture of real-time network data. Through this mechanism, crucial attributes such as SYN size and window size are extracted from intercepted packets. These attributes serve as the input features for a Random Forest Classifier, a machine learning model trained on an extensive dataset encompassing diverse OS fingerprints.

Complementing the passive approach, our framework incorporates an active fingerprinting module that engages in manual TCP connection establishment. This proactive strategy involves scanning open ports on each IP address within the network, providing a more comprehensive analysis in scenarios where passive techniques may be constrained. Similar to the passive component, the active module utilizes the same machine learning model for OS prediction, ensuring consistency in results.

At the heart of our framework lies the Random Forest Classifier, implemented using the scikit-learn library. This machine learning model demonstrates exceptional accuracy in predicting OS based on features extracted from both passive and active fingerprinting. Practical implementation is realized through a set of Bash scripts orchestrating the execution of Python modules. Users are afforded the flexibility to choose between passive and active fingerprinting methods based on their specific network assessment requirements.

The tool's adaptability is further demonstrated by its ability to seamlessly integrate with live network traffic, using PyShark to capture and analyze packets in real-time. This paper unfolds a new chapter in OS fingerprinting, bridging traditional and modern approaches to fortify cybersecurity measures in contemporary network environments.

II. DATASET AND FEATURE SELECTION

A. The Dataset Dilemma

The quest for an empirically robust and diverse dataset, encompassing a spectrum inclusive of IoT devices, embarked

upon a meticulous review of seminal research papers. Within the corpus of identified datasets, a comprehensive exploration unfolded across three seminal research papers:

1. *A Machine-Learning-Based Tool for Passive OS Fingerprinting* (encompassing 4 datasets)

2. *Passive Operating System Fingerprinting Revisited: Evaluation and Current Challenges* (encompassing over 50 datasets, curated from prior OS fingerprinting research papers, presented chronologically)

3. *Passive OS Fingerprinting Methods in the Jungle of Wireless Networks* (inclusive of 3 datasets sourced from university network flows)

This scrutiny surfaced challenges inherent in extant public datasets, ranging from the ubiquitous absence of recorded major and minor OS versions to the temporal obsolescence of nearly 37 datasets, rendering the OS versions incongruent with the contemporary timeframe of our research. Further, the paucity of online availability, attributed to the sensitive nature of the data and its potential for malicious exploitation, necessitated a judicious dataset curation approach.

After a meticulous discernment process, our focus converged onto select datasets:

1. *Masaryk University (Finger_map)* (derived from Paper 3)
2. *Masaryk University (Finger_database)* (extracted from Paper 3)
3. *University of Houston College Network Dataset* (accessed from GitHub)

B. Feature Selection

In the context of the designated datasets (1, 2, and 3), the feature exploration phase was marked by systematic experimentation with permutations of TCP SYN size, TCP window size, and Time to Live (TTL). The rationale underpinning the selection of these features resides in their intrinsic technical significance:

- **Initial SYN Packet Size (synSize):** This feature encapsulates the nuanced intricacies of the OS's implementation, serving as a discerning fingerprint identifier.
- **TCP Window Size (winSize):** Representing the initial window size, this feature delves into the receiving side's readiness, dictated by OS settings.
- **Time to Live (TTL):** As an IPv4 protocol value initiated by the OS kernel, TTL plays a pivotal role in averting perpetual packet routing.

The chosen features for datasets 1, 2, and 3 encapsulate critical facets of network communication intricately intertwined with OS idiosyncrasies, affording a granular comprehension of the OS fingerprinting paradigm. In the forthcoming analytical discourse on model performance, we shall explicate the

outcomes derived from each dataset, elucidating the rationale that precipitated the definitive selection of our ultimate dataset for the OS fingerprinting tool.

C. Dataset Preprocessing

Effective dataset preprocessing is paramount to ensure the integrity and reliability of the subsequent machine learning model. In the context of our OS fingerprinting tool, the preprocessing pipeline is characterized by meticulous steps aimed at refining and purifying the dataset for optimal model training and evaluation.

1. Removal of Entries without Major and Minor Version Records:

Given the significance of major and minor OS versions in our feature set, entries lacking these crucial records were systematically excised from the dataset. This strategic pruning ensures that only instances with comprehensive OS information contribute to the model training process. Such a meticulous approach guards against potential inaccuracies and enhances the model's precision in OS prediction.

2. Exclusion of Outdated OS Versions:

Acknowledging the temporal relevance required for accurate OS prediction, occurrences of outdated OS versions were systematically removed from the dataset. This measure addresses the potential skewing effect obsolete versions might have on the model's performance. The curation of a contemporary dataset is pivotal in aligning the model with the current OS landscape, facilitating more accurate predictions.

3. Handling Missing Values:

To bolster the dataset's completeness, any instances with missing values were judiciously handled. This involved the removal of rows where essential features crucial for OS prediction were not available. This meticulous handling of missing values ensures that the model is trained on a dataset characterized by completeness and coherence.

III. MODEL ARCHITECTURE AND PERFORMANCE

A. Testing different models on datasets

For a comprehensive understanding of dataset behaviors, personalized models employing various machine learning classification algorithms were employed.

Classifier	Accuracy	Precision	Recall	F1
KNN	0.634	0.546	0.634	0.582
SVM	0.341	0.295	0.341	0.252
Random Forest	0.780	0.726	0.780	0.744
AdaBoost	0.683	0.654	0.683	0.656
Gradient Boosting	0.707	0.637	0.707	0.661
Decision Tree	0.732	0.656	0.732	0.682
Logistic Regression	0.317	0.240	0.317	0.268
Naive Bayes	0.683	0.664	0.683	0.665
ANN	0.610	0.451	0.610	0.512

Table. 1. Dataset 1

Classifier	Accuracy	Precision	Recall	F1
KNN	0.443	0.396	0.443	0.388
SVM	0.381	0.146	0.381	0.211
Random Forest	0.488	0.349	0.488	0.396
AdaBoost	0.405	0.279	0.405	0.330
Gradient Boosting	0.485	0.348	0.485	0.395
Decision Tree	0.488	0.349	0.488	0.396
Logistic Regression	0.381	0.146	0.381	0.211
Naive Bayes	0.089	0.356	0.089	0.099
ANN	0.460	0.275	0.460	0.344

Table. 2. Dataset 2

Classifier	Accuracy	Precision	Recall	F1
KNN	0.847	0.844	0.847	0.842
SVM	0.523	0.625	0.523	0.407
Random Forest	0.914	0.912	0.914	0.912
AdaBoost	0.501	0.302	0.501	0.377
Gradient Boosting	0.815	0.843	0.815	0.799
Decision Tree	0.909	0.907	0.909	0.907
Logistic Regression	0.429	0.397	0.429	0.400
Naive Bayes	0.357	0.485	0.357	0.389
ANN	0.497	0.456	0.497	0.440

Table. 3. Dataset 3

B. Dataset Performance and Model:

- **Dataset 3:** While exhibiting high accuracy, indications of potential overfitting and an aging OS class distribution led to its exclusion.
- **Dataset 2:** Consistently underwhelming performance across classifiers signaled its inadequacy for the tool's objectives.
- **Dataset 1:** Despite moderate accuracy, the Random Forest classifier yielded the best results at **78.05%** accuracy, **72.6%** precision and **78%** recall after eliminating TTL as a feature. Dataset 1, being contemporary and demonstrating superior generalization, emerged as the preferred choice.
- The Random Forest classifier proved optimal for OS fingerprinting, excelling in handling class imbalances and mitigating noise. The model's versatility, coupled with its capacity to handle imbalanced datasets and reduce overfitting, makes it a robust choice for nuanced OS prediction in dynamic network environments.

Accuracy Increase by Dropping a Feature Observation:

A noteworthy observation was made in Dataset 1, where the removal of TTL as a feature led to a significant 10% increase in accuracy, underscoring the importance of meticulous feature selection.

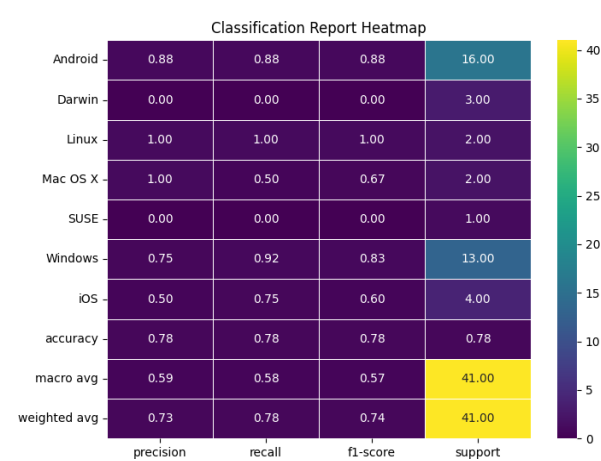
C. Observations of Actual Results versus Research Paper Results:

The tool consistently yielded lower results compared to reported outcomes in the research papers, indicating the

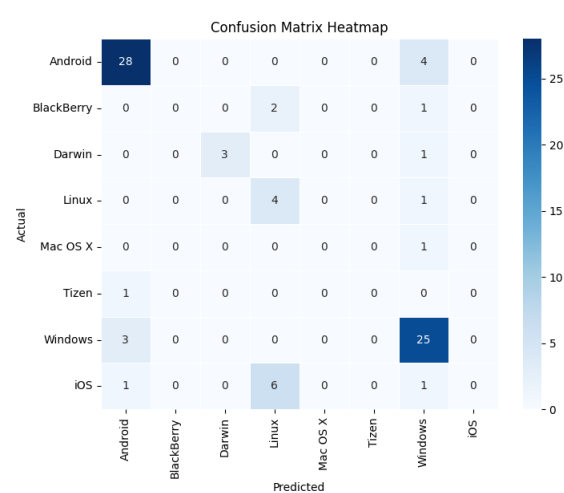
nuanced nature of real-world application and the need for tailored adjustments. It also raised doubts about the legitimacy of the reported results.

Classifier	Accuracy	Precision	Recall
SVM	Reported:82.17% Actual:34.1%	Reported:0.82 Actual:0.295	Reported:0.82 Actual:0.341
RF	Reported:84.76% Actual:78.05%	Reported:0.85 Actual:0.726	Reported:0.84 Actual:0.780
KNN	Reported:84.28% Actual:63.4%	Reported:0.85 Actual:0.546	Reported:0.83 Actual:0.634

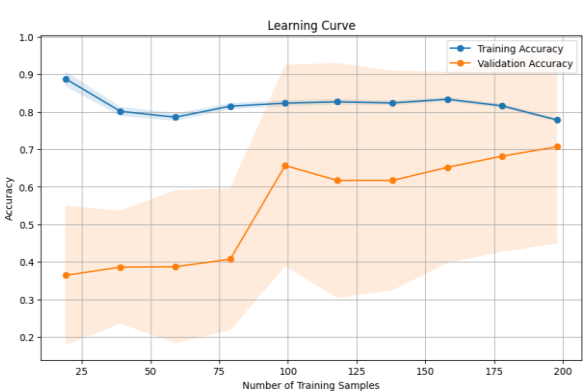
D. Classification Report Heatmap



E. Confusion Matrix

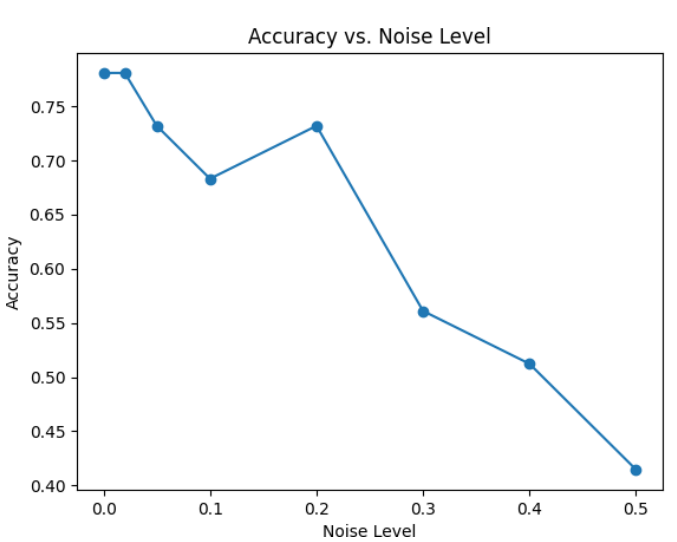


F. Learning Curve



In the learning curve, a notable trend emerges as the number of training samples increases: the convergence of the validation accuracy line towards the training accuracy line. Initially, any disparity between these lines may signal challenges in model generalization or potential overfitting. However, with an expanding training set, the validation accuracy line gradually approaches the training accuracy line, indicating improved performance on previously unseen data. This convergence implies that the model benefits from additional training samples, capturing nuanced patterns while maintaining a delicate balance between complexity and generalization. The diminishing gap signifies a model that adeptly adapts to increasing dataset complexity without compromising its ability to generalize—a hallmark of a well-performing model resilient to overfitting and underfitting.

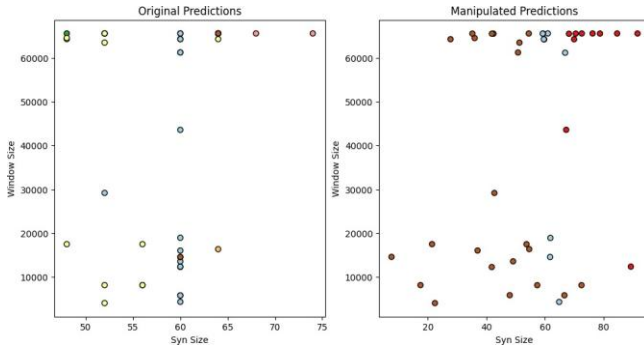
G. Model Resilience Amidst Noise



In this iterative exploration, we progressively augment the percentage of noise within the training dataset, closely monitoring the fluctuation in the model's accuracy. Accompanying this investigation is a graph that visually captures the nuanced relationship between noise levels and model performance. Notably, the model exhibits robustness to moderate noise, maintaining stable accuracy. However, an intriguing observation emerges as noise exceeds 20%—the model experiences sudden dips in accuracy, signaling a potential threshold beyond which the impact of noise becomes more pronounced.

H. Impact of Deliberate Feature Manipulation

In this study, our focus was to assess the resilience of our OS fingerprinting model in the face of deliberate feature manipulation, a potential tactic employed by adversaries to deceive detection systems. By systematically manipulating features in the testing set and adjusting synthesis and window size factors, we observed a substantial decline in model accuracy. Notably, with a manipulation factor **set to 20**, the accuracy of the OS fingerprinting model dropped to around **40%**. These findings underscore the importance of fortifying intrusion detection systems against intentional feature tampering for reliable and effective real-world application.



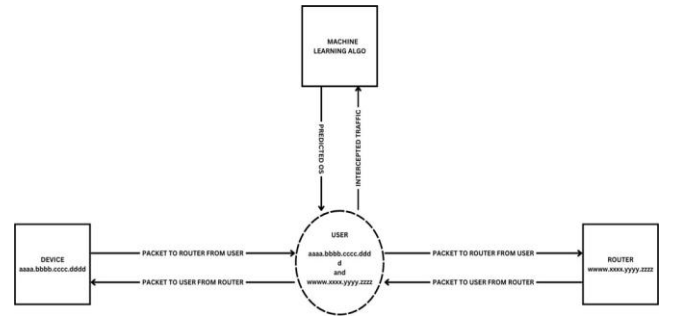
V. MODEL DEPLOYMENT

A. Implementation

- We have utilized ARP spoofing and TCP connections to gather the necessary data required for feeding into the machine learning algorithm.
- Ideally, we use ARP spoofing to capture TCP packets on the network in real-time and actively feed them into the machine learning algorithm. However, if a situation arises where we are unable to capture TCP packets through spoofing, we can establish a TCP connection with the target device to generate a TCP packet, albeit at the cost of our anonymity.

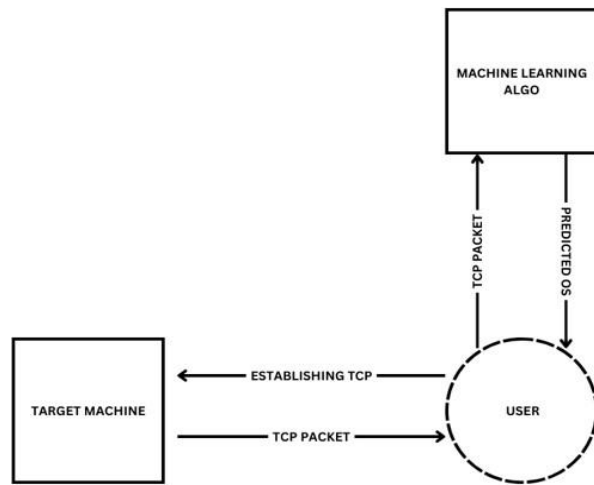
B. ARP Spoofing

- In our tool, we have two methods for OS fingerprinting. One of them is ARP spoofing. In ARP spoofing, we first spoof the attacker device to be all the different IP addresses on the network, which results in our device receiving all the packets that are transmitted over the network. We now take these received packets and give them as input to the machine learning algorithm that classifies the TCP packets (if any) and returns the predicted operating system of the devices on our network.
- Using this method, we are not exposed to the devices as our identity is hidden. We use a real-time approach for this tool so that as we receive TCP packets, they are fed into the machine learning algorithm and provide the classified operating system.



C. TCP Connection

- The other method we have in our tool to find the operating system is through TCP connection. Sometimes, we may not receive TCP packets from devices we want to classify through ARP spoofing because the TCP packet does not exist. In this case, we have to generate a TCP packet from the device onto the network. To do this, we need to first establish a TCP connection with this particular device.
- Connecting to a device through TCP will return us a TCP packet. We can then use this packet as input into our algorithm to classify the operating system of the particular device. However, when we use this method, the device will know that we have connected to it, and we are receiving TCP packets.



B. How to setup the project?

- Step 1: we download the program from the git repository we unzip it
- Step 2: open a terminal in the same directory and give command **sudo ./main.sh <interface_name> <mode(-a/-p)>**
- Enter the specific interface name at <interface_name>
- Choose between the two available modes -a for ARP and -p for TCP.

VI. TEST EXPERIMENT

A. Goal

- Our goal is to scan our test network (TP-link - IoT lab)
- We should have 4 devices returning with their respective operating systems namely
 - 1) router
 - 2) printer
 - 3) camera
 - 4) Users wifi card

B. Experiment

- Enter the specific interface name at <interface_name> we use wlo1.
- Choose between the two available modes -a for ARP and -p for TCP.
- We go for -p which uses ARP spoofing, as we can see below we do not get any TCP packets.

```

root@fedora:/home/johnambooken/final project
File Edit View Search Terminal Help
Accuracy on the test set: 78.05%
Classification Report:
precision    recall  f1-score   support

   Android    0.88    0.88    0.88        16
    Darwin    0.00    0.00    0.00         3
     Linux    1.00    1.00    1.00         2
  Mac OS X    1.00    0.50    0.67         2
     SUSE     0.00    0.00    0.00         1
    Windows    0.75    0.92    0.83        13
       iOS    0.50    0.75    0.60         4

 accuracy          0.59          0.58          0.57          41
  macro avg          0.59          0.58          0.57          41
weighted avg          0.73          0.78          0.74          41

['192.168.1.103', '192.168.1.100', '192.168.1.107', '192.168.1.1']
*****
192.168.1.103:80 TCP connection established.
Predicted OS: Linux, Predicted Major Version: 13.0, Predicted Minor Version: 2.0
*****
192.168.1.100:443 TCP connection established.
Predicted OS: Linux, Predicted Major Version: 13.0, Predicted Minor Version: 2.0
*****
192.168.1.107:5355 TCP connection established.
Predicted OS: Linux, Predicted Major Version: 13.0, Predicted Minor Version: 2.0
*****
192.168.1.1:22 TCP connection established.
Predicted OS: Linux, Predicted Major Version: 13.0, Predicted Minor Version: 2.0
*****
  
```

- To get TCP packets we go for a direct TCP connection here. And as we can see below we see four devices on the network classified

```

root@fedora:/home/johnambooken/final project
File Edit View Search Terminal Help
Accuracy on the test set: 78.05%
Classification Report:
precision    recall  f1-score   support

   Android    0.88    0.88    0.88        16
    Darwin    0.00    0.00    0.00         3
     Linux    1.00    1.00    1.00         2
  Mac OS X    1.00    0.50    0.67         2
     SUSE     0.00    0.00    0.00         1
    Windows    0.75    0.92    0.83        13
       iOS    0.50    0.75    0.60         4

 accuracy          0.59          0.58          0.57          41
  macro avg          0.59          0.58          0.57          41
weighted avg          0.73          0.78          0.74          41

['192.168.1.103', '192.168.1.100', '192.168.1.107', '192.168.1.1']
*****
192.168.1.103:80 TCP connection established.
Predicted OS: Linux, Predicted Major Version: 13.0, Predicted Minor Version: 2.0
*****
192.168.1.100:443 TCP connection established.
Predicted OS: Linux, Predicted Major Version: 13.0, Predicted Minor Version: 2.0
*****
192.168.1.107:5355 TCP connection established.
Predicted OS: Linux, Predicted Major Version: 13.0, Predicted Minor Version: 2.0
*****
192.168.1.1:22 TCP connection established.
Predicted OS: Linux, Predicted Major Version: 13.0, Predicted Minor Version: 2.0
*****
  
```

C. Result

- The devices are classified.
- The devices know that we have received TCP packets.

VII. LIMITATIONS

1. Passive OS Fingerprinting Dependency:

The passive component relies on devices actively sending TCP packets. In the absence of such communication, OS prediction becomes infeasible. To overcome this, the active tool can be employed to manually establish a TCP connection.

2. Active OS Fingerprinting Speed:

The exhaustive port scanning process in the active method, spanning ports 1 to 65535 for each IP, introduces a significant time overhead. This slowness may impact real-time applicability, especially in large-scale networks.

3. Open Port Requirement:

The effectiveness of active OS fingerprinting depends on open ports. If ports are closed or behind a firewall, the tool cannot establish a TCP connection, limiting OS prediction capabilities.

4. Resource Intensiveness:

Active OS fingerprinting, particularly in extensive IP scans, demands substantial computational and network resources, potentially limiting scalability in resource-constrained environments. Understanding these limitations is crucial for interpreting results in diverse network scenarios.

conclusion

```
ws4_request&X-Amz-Signature=59ba0334c2778650d88c1c46c8ff68b3aa9141
2b724608136347a6305b50708a&hash=f88d628bfbfca09d77a0300a2d9defe
1812162c7751022b82cbl0d9470332ca&host=68042c943591013ac2b2430a89
b270f6af2c76d8df0886a07176afe7c76c261&pii=S138912862300227X&tid=
spdf879a9c04cadf4758-9434-b0b9d58d934dc&sid=bc8079a791973749da-b6
11-b5e766fa91lexgrb&type=client&tsod=d3d31NnJjaWVWuYVtY3k4XJlY3Qu
Y29t&ua=1b055a51520401595458&r=834ce84949ad4c69&cc=ae
```

```
ws4_request&X-Amz-Signature=59ba0334c2778650d88c1c46c8ff68b3aa9141
2b724608136347a6305b50708a&hash=f88d628b6fbfca097d77a0300a2d9defe
1812162c7751022b82cb1d09470332cae&host=68042c943591013ac2b2430a89
b270f6af2c76d8df0886a071716afe7c76c261&pii=S138912862300227X&tid=
spdf879a9c04cadf4758-9434-b0b09d58d934d&ssid=bc8079a971973749da-b6
11-b5e766fa91egxqr&type=client&tsod=d3d31NnJjaWVWuYVY917vkaXJlY3Qu
Y29t&ua=1b055a51520401595458&r=834ce84949ad4c69&cc=ae
```

```
ws4_request&X-Amz-Signature=59ba0334c2778650d88c1c46c8ff68b3aa9141
2b724608136347a6305b50708a&hash=f88d628b6fbca097d77a0300a2d9defe
1812162c7751022b82cbl0d9470332cae&host=68042c943591013ac2b2430a89
b270f6af2c76d8df0886a071716afe7c76c261&pii=S138912862300227X&tid=
spdf879a9c04cadf4758-9434-b0b9d58d934dc&sid=bc8079a791973749da-b6
11-b5e766fa91lexgrb&type=client&tsod=d3d31NnJjaWVwYVY2V1Y3kaXJlY3Qu
Y29t&ua=1b055a151520401595458&r=834ce84949ad4c69&cc=ae
```

```
ws4_request&X-Amz-Signature=59ba0334c2778650d88c1c46c8ff68b3aa9141
2b724608136347a6305b50708a&hash=f88d628b6fbfca097d77a0300a2d9defe
1812162c7751022b82cbl0d9470332ca&host=68042c943591013ac2b2430a89
b270f6af2c76d8df0886a071716afe7c76c261&pii=S138912862300227X&tid=
spdf879a9c04cadf4758-9434-b0b09d58d9d4d&sid=bc8079a971973749da-b6
11-b5e766fa91lexgrb&type=client&tsod=33d31NnJiaWVvUyV917k4XJlY3Qu
Y29t&ua=1b055a51520401595458&r=834ce84949d4d69f&cc=ae
```

```
ws4_request&X-Amz-Signature=59ba0334c2778650d88c1c46c8ff68b3aa9141
2b724608136347a6305b50708a&hash=f88d628b6fbfca097d77a0300a2d9defe
1812162c7751022b82cb1d09470332cae&host=68042c943591013ac2b2430a89
b270f6af2c76d8df0886a071716afe7c76c261&pii=S138912862300227X&tid=
spdf879a9c04cadf4758-9434-b0b09d58d934d&ssid=bc8079a971973749da-b6
11-b5e766fa91egxqr&type=client&tsod=d3d31NnJjaWVWuYVY917vkaXJlY3Qu
Y29t&ua=1b055a51520401595458&r=834ce84949ad4c69&cc=ae
```

- ```
ws4_request&X-Amz-Signature=59ba0334c2778650d88c1c46c8ff68b3aa9141
2b724608136347a6305b50708a&hash=f88d628b6fbfca097d77a0300a2d9defe
1812162c7751022b82cbl0d9470332ca&host=68042c943591013ac2b2430a89
b270f6af2c76d8df0886a071716afe7c76c261&pii=S138912862300227X&tid=
spdf879a9c04cadf4758-9434-b0b09d58d9d4d&sid=bc8079a971973749da-b6
11-b5e766fa91lexgrb&type=client&tsod=33d31NnJiaWVvUyV917k4XJlY3Qu
Y29t&ua=1b055a51520401595458&r=834ce84949d4d69f&cc=ae
```